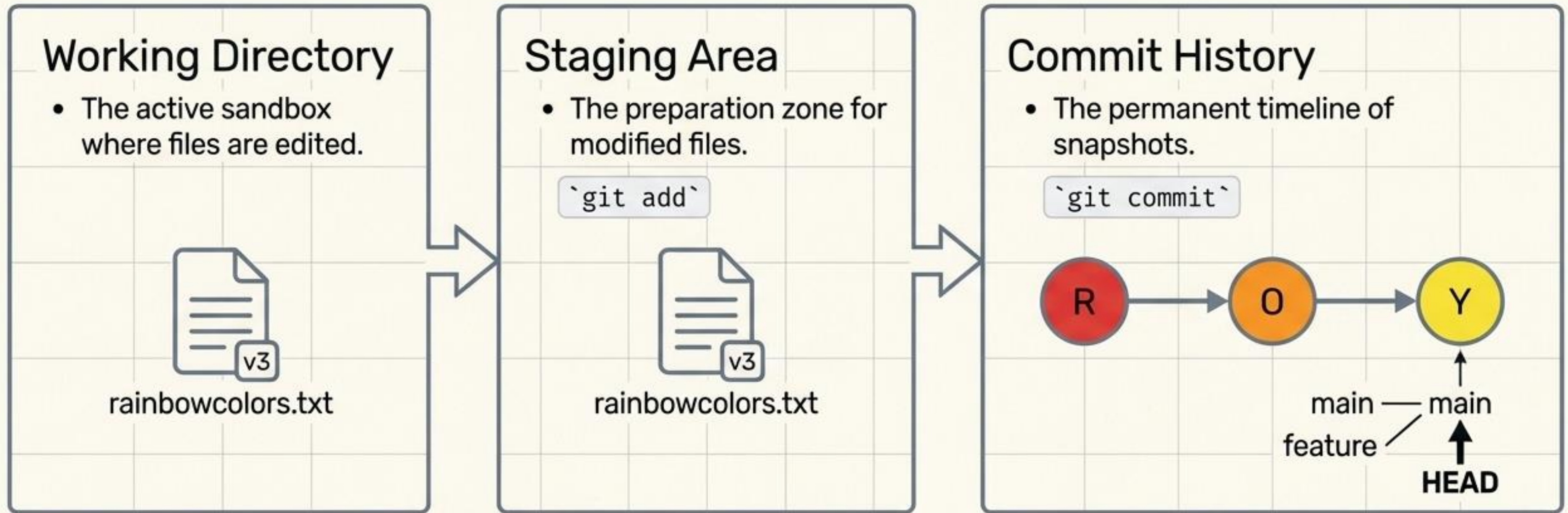


Recap: The Solitary Version Control Loop



The Limitation: This standard loop creates a single, rigid line of development.

Breaking Out of the Single Timeline

The Single-Line Problem

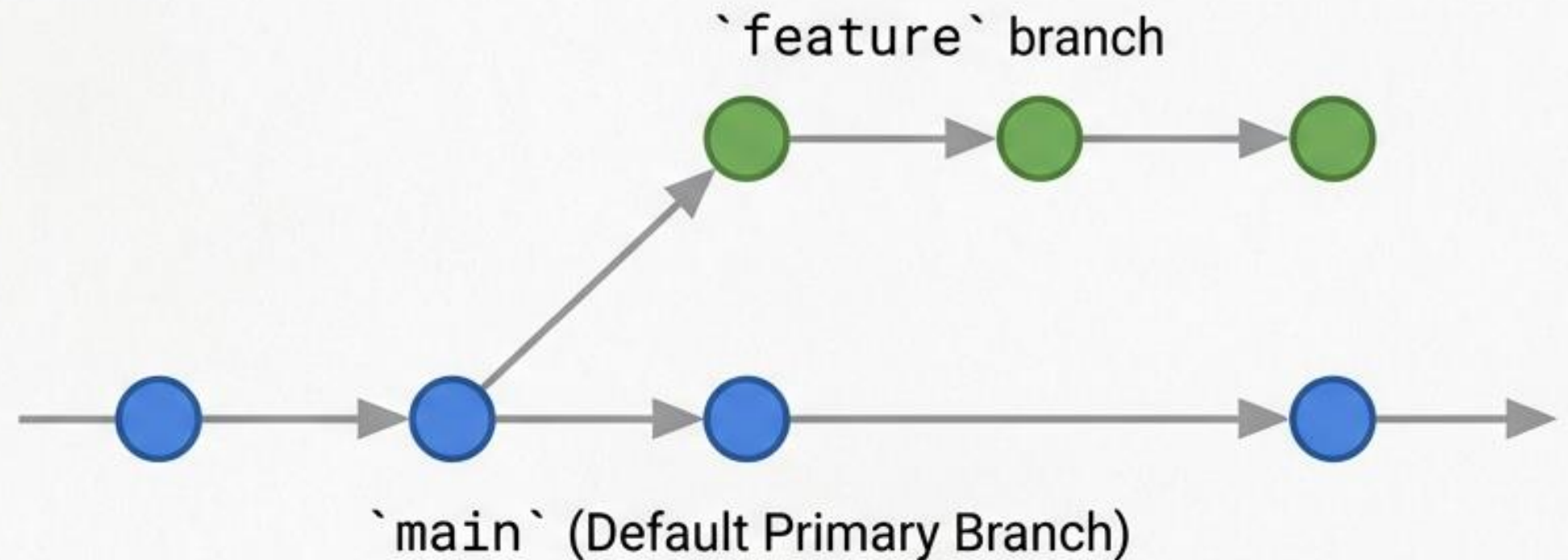
Working directly on the main project risks breaking stable code and prevents parallel teamwork.

The Branch Solution

Branches are independent lines of development.

The Purpose

Safely test new features or fix bugs in isolated 'parallel universes' before integrating them.



Branches Are Movable Pointers, Not Copies

- **Lightweight References:**

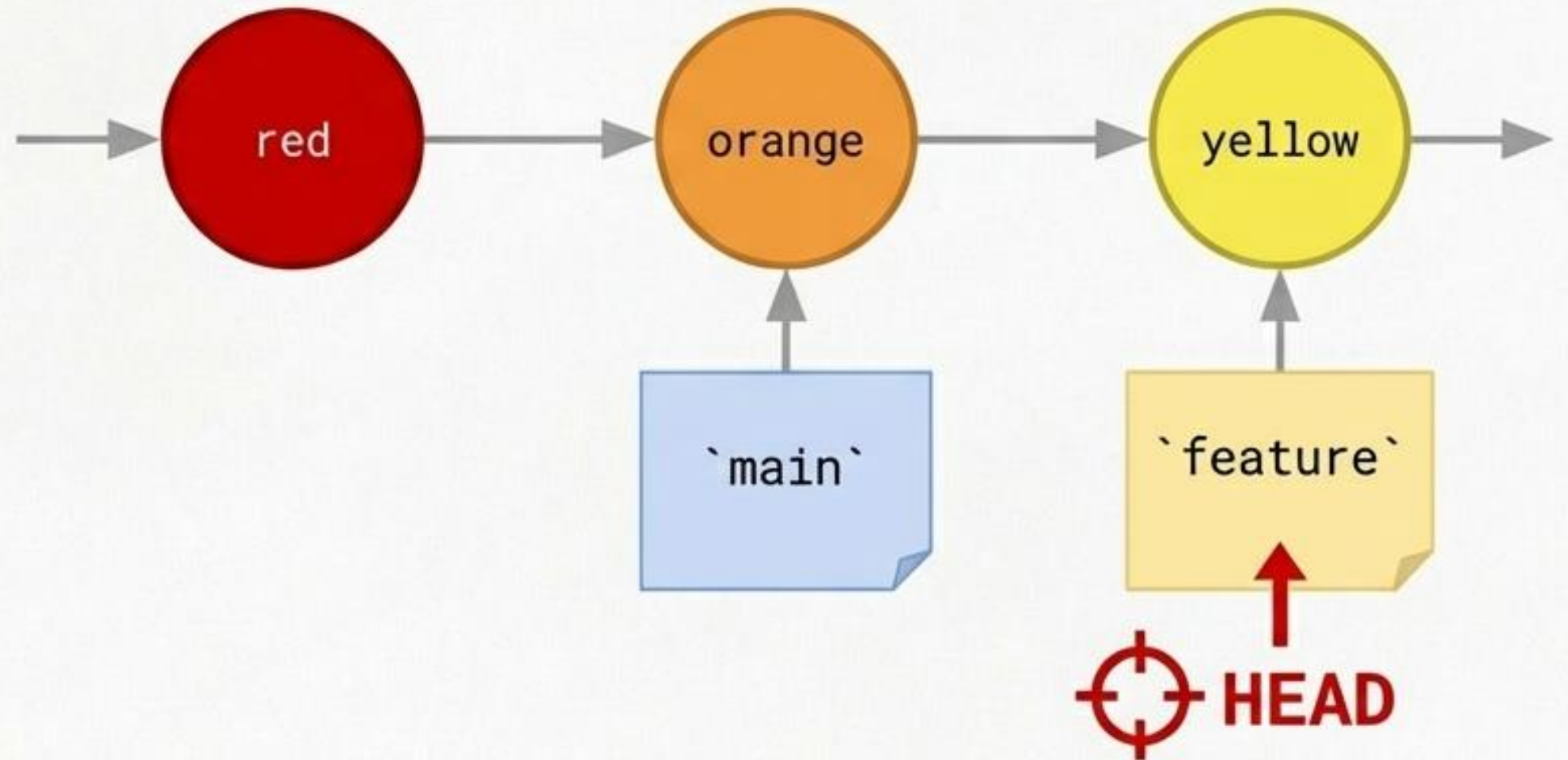
A branch is simply a 41-byte file containing a commit hash, not a duplicate folder.

- **The Sticky Note Metaphor:**

Think of a branch as a movable sticky note pointing to a specific commit node.

- **The HEAD Pointer:**

A special target indicator showing exactly 'You are here'.

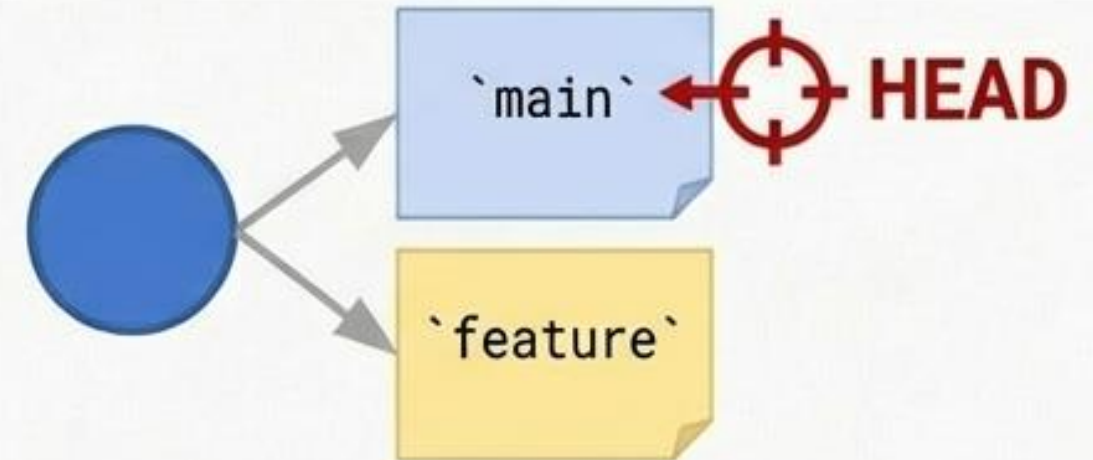


Navigating the Multiverse: Creating and Switching

Action: Create a Branch

```
`git branch <branch_name>`
```

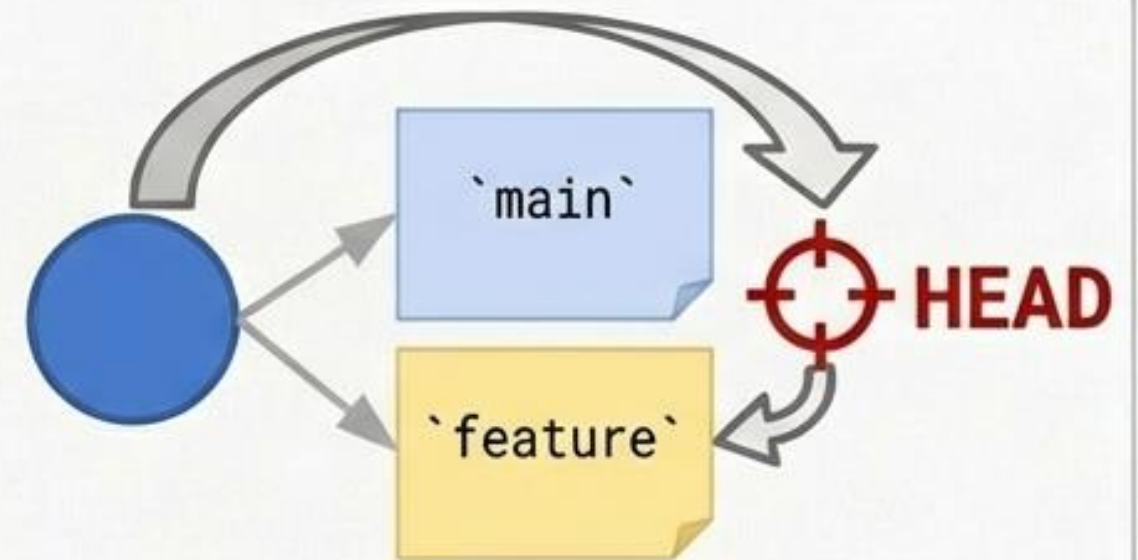
Generates a new pointer at the current commit.



Action: Switch Branches

```
`git switch <branch_name>`
```

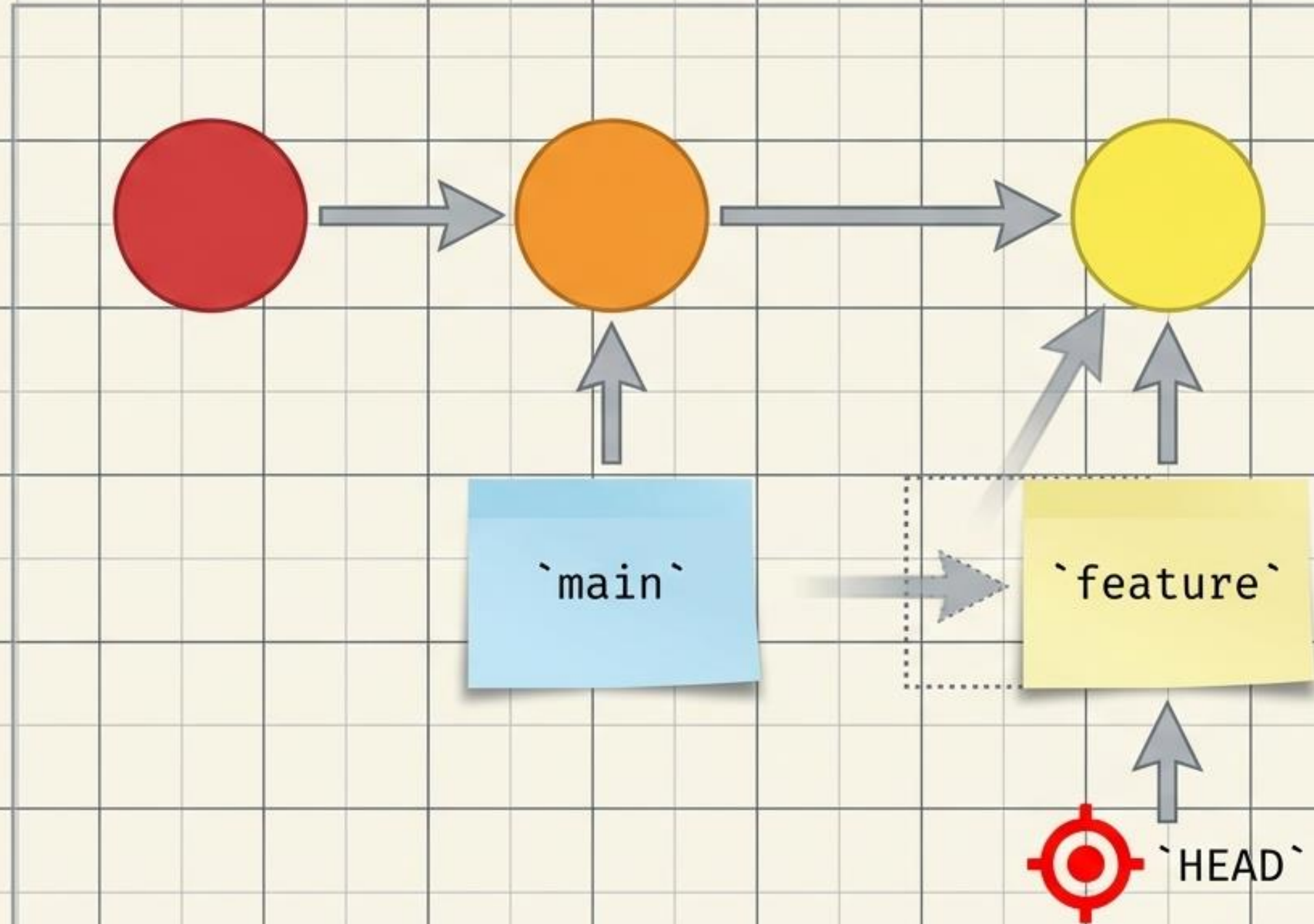
Moves the HEAD target to the specified branch. The Working Directory instantly updates to match.



Shortcut: ``git switch -c <branch_name>`` creates and switches in one step.

Independent Evolution on a Branch

- Commits attach to the currently active branch (where HEAD points).
- Adding a new commit pushes the active branch pointer forward.
- The previous branch pointer remains untouched on the older commit.



The Great Convergence: Merging Branches

`git merge <source_branch>` integrates changes into your currently active target branch.

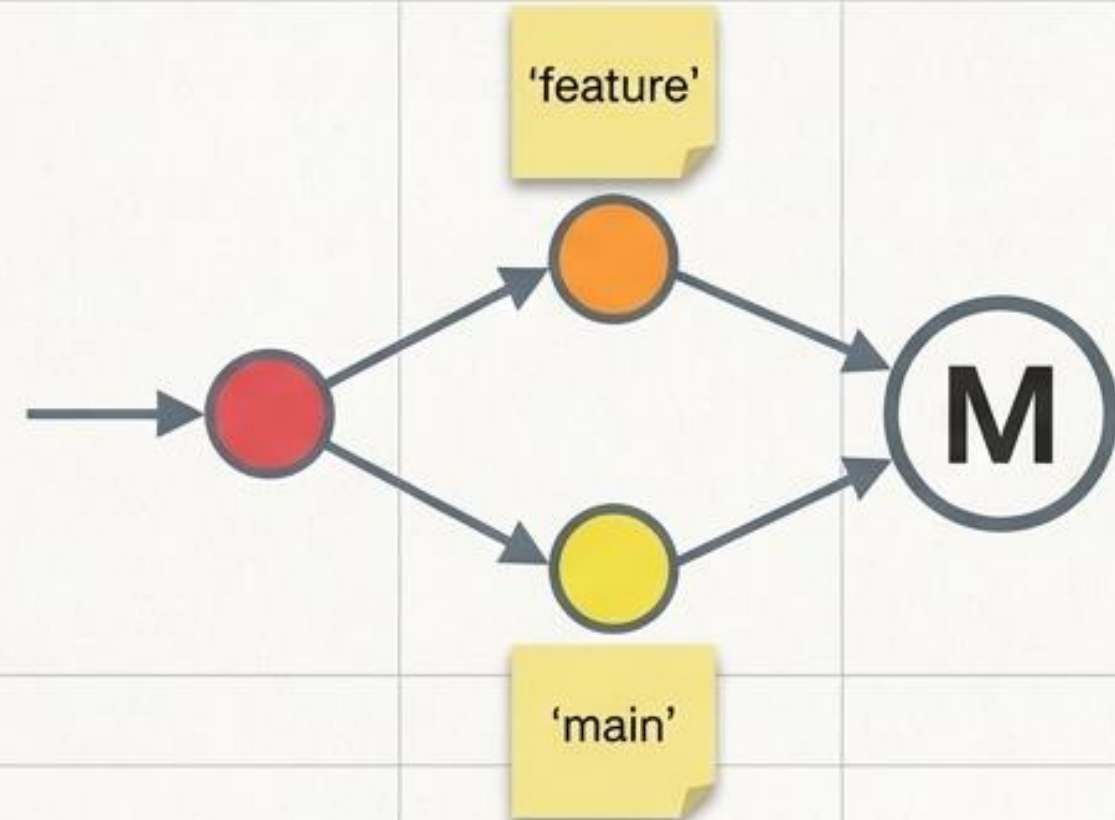
Condition 1: Fast-Forward Merge

History has not diverged. Target branch simply moves its pointer forward.

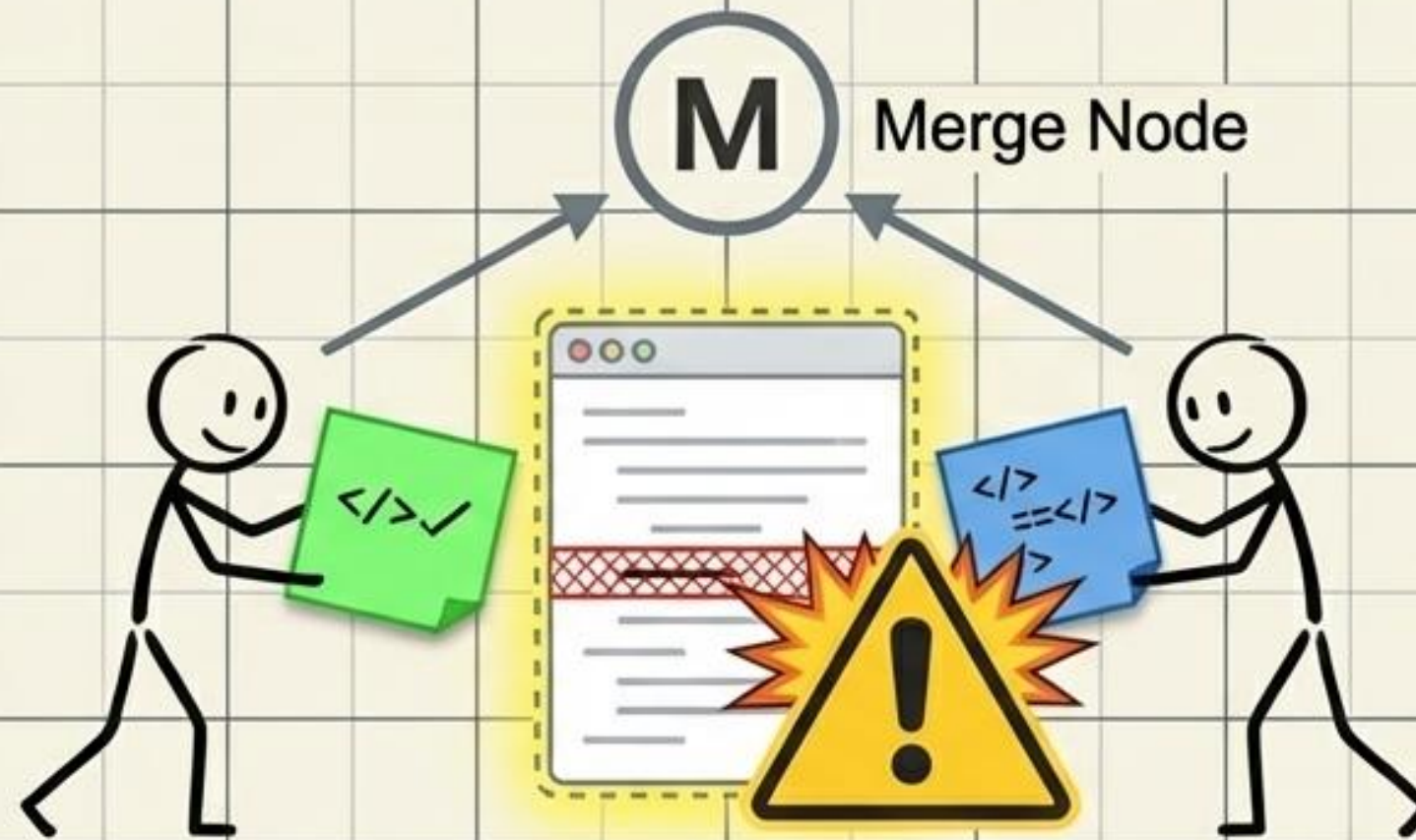


Condition 2: Three-Way Merge

History has diverged. Git creates a distinct new 'Merge Commit' to tie histories together.



When Timelines Collide: Merge Conflicts



The Cause

Two divergent branches edited the exact same line of code, or one deleted a file while the other modified it.

The Halting Point

Git pauses the three-way merge. It refuses to blindly overwrite your work.

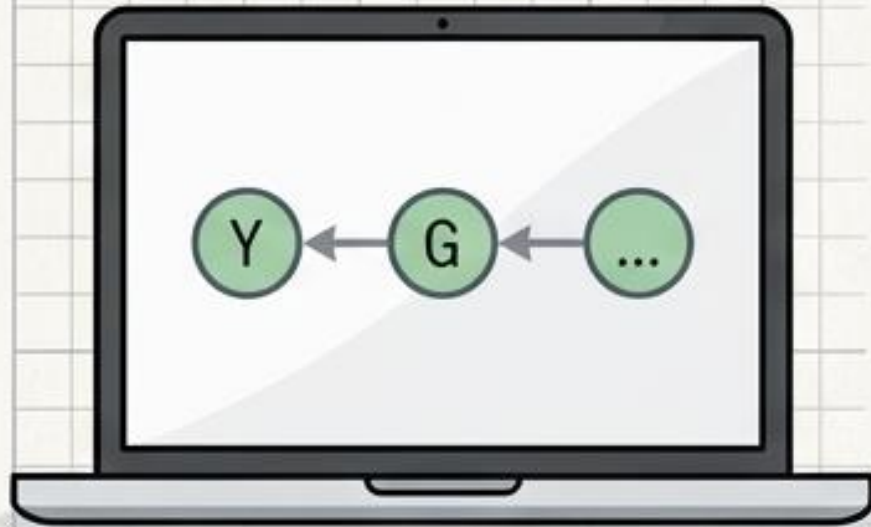
The Resolution

The developer manually edits the file to choose the correct code, stages the file (`git add`), and commits to finish.

Git vs. GitHub: The Local Engine and The Global Hub

Remote Repositories: A version of your project hosted on the internet, serving as the definitive ‘source of truth’.

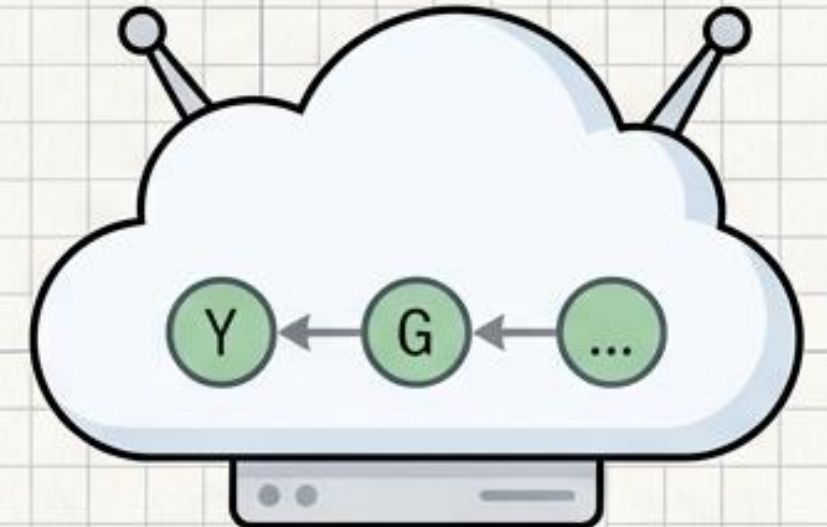
The Engine: Local (Git)



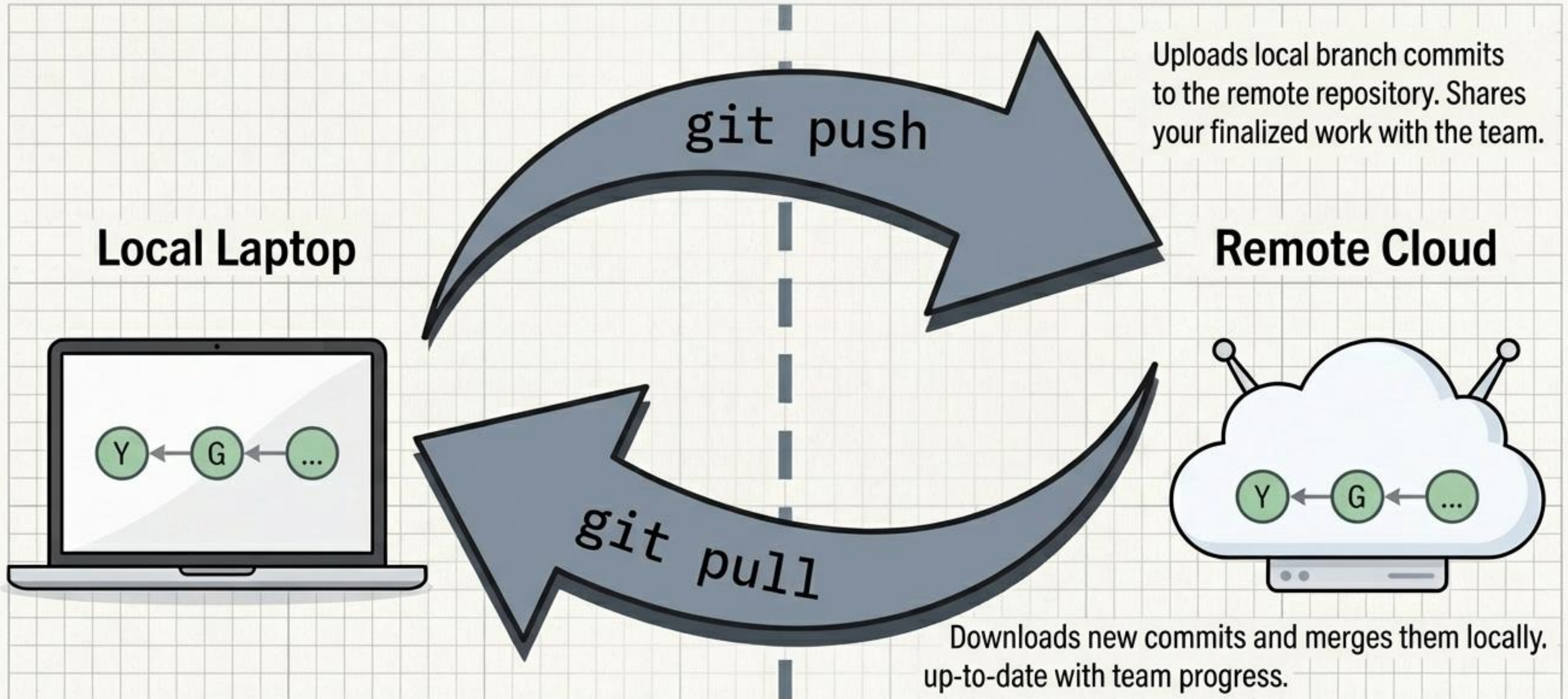
- Command-line tool
- Personal version history sandbox
- Completely offline

The Hub: Remote (GitHub / origin)

- Cloud hosting service
- Provides GUI
- Centralized team collaboration



Synchronizing the Repositories



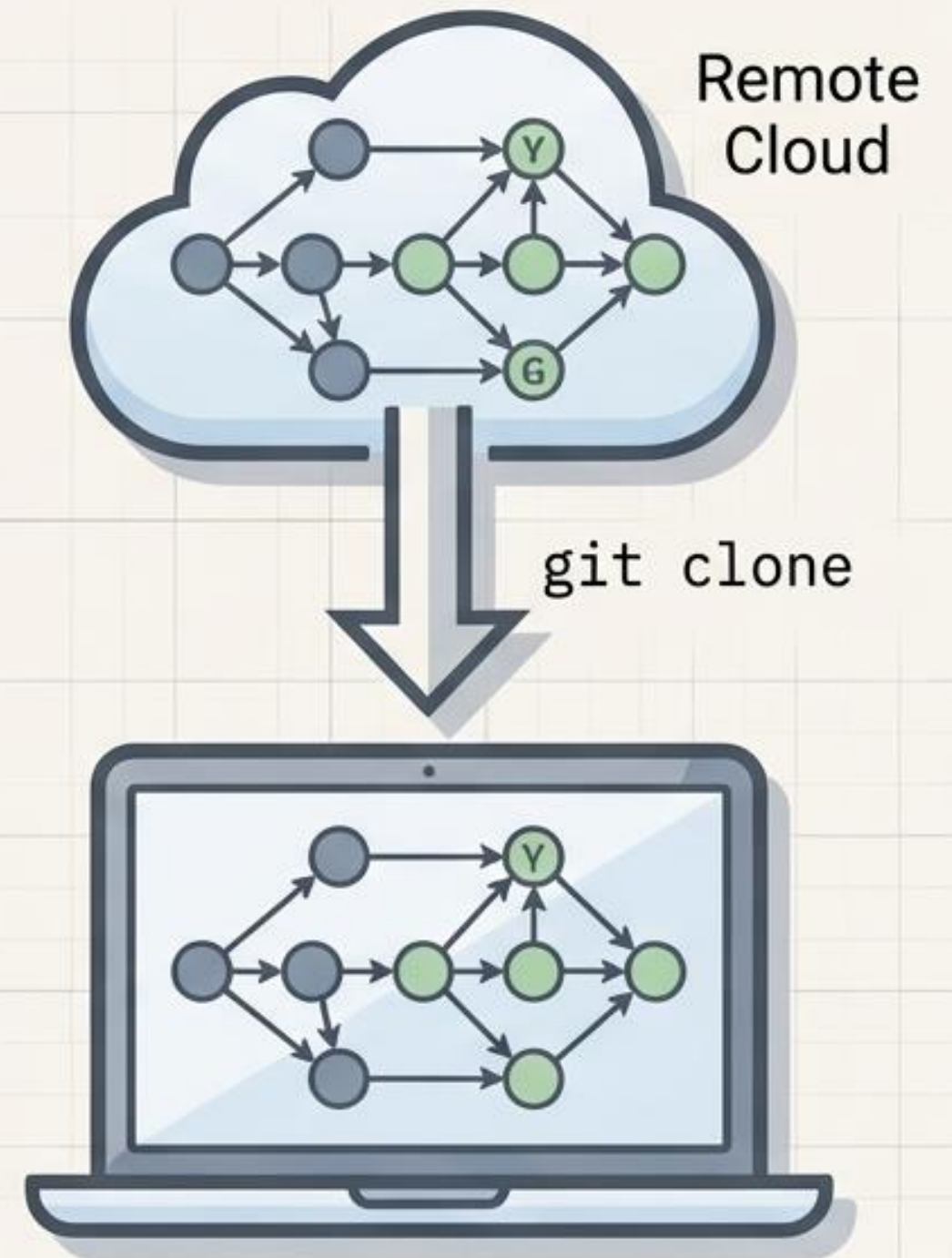
Cloning: Adopting a Remote Project

The Action: `git clone <url>`

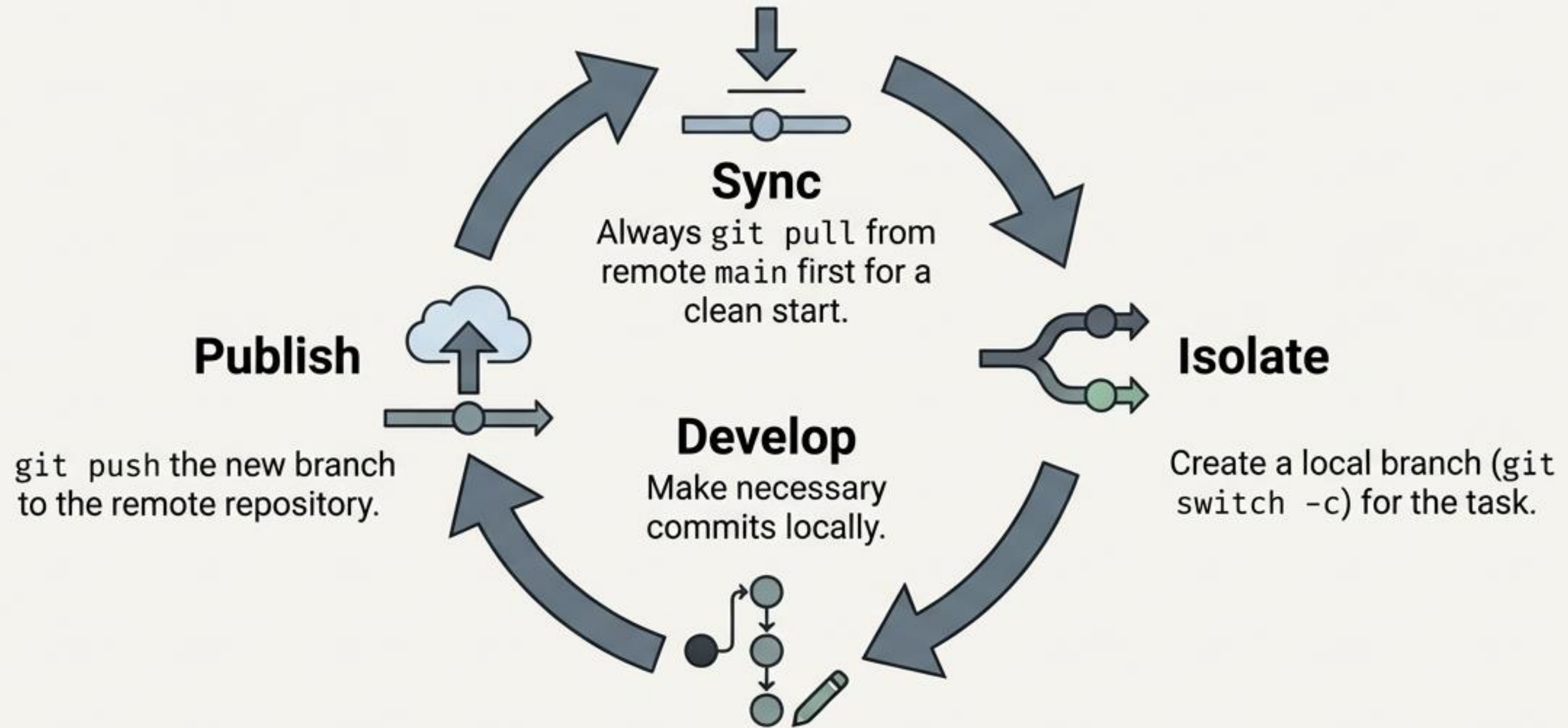
The Result: Downloads a complete, exact copy of a remote repository onto a local machine.

What is Included: Not just the files, but the entire chronological history, all commit nodes, and tracking branches.

The Connection: Automatically links the new local repository to the remote source (origin).



The Professional Collaboration Cycle



The Golden Rule: Never commit directly to the shared remote production branch (`main`).

Pull Requests & The Code Review Checkpoint

What is a PR?

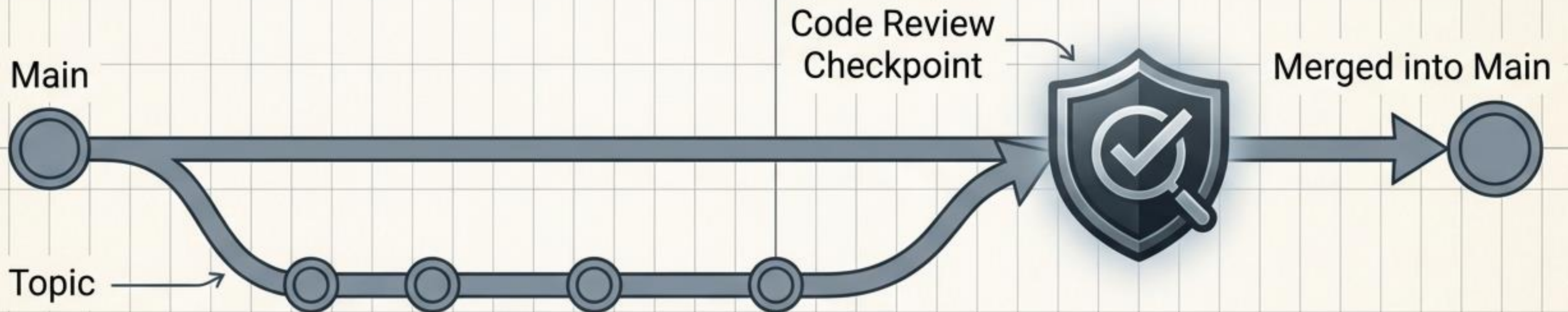
A formal request on GitHub to merge your completed remote branch into the `main` branch.

The Code Review

Team members inspect code, leave feedback, and request updates.

The Approval

Once tests pass and reviewers approve, the PR is merged remotely, protecting stability.



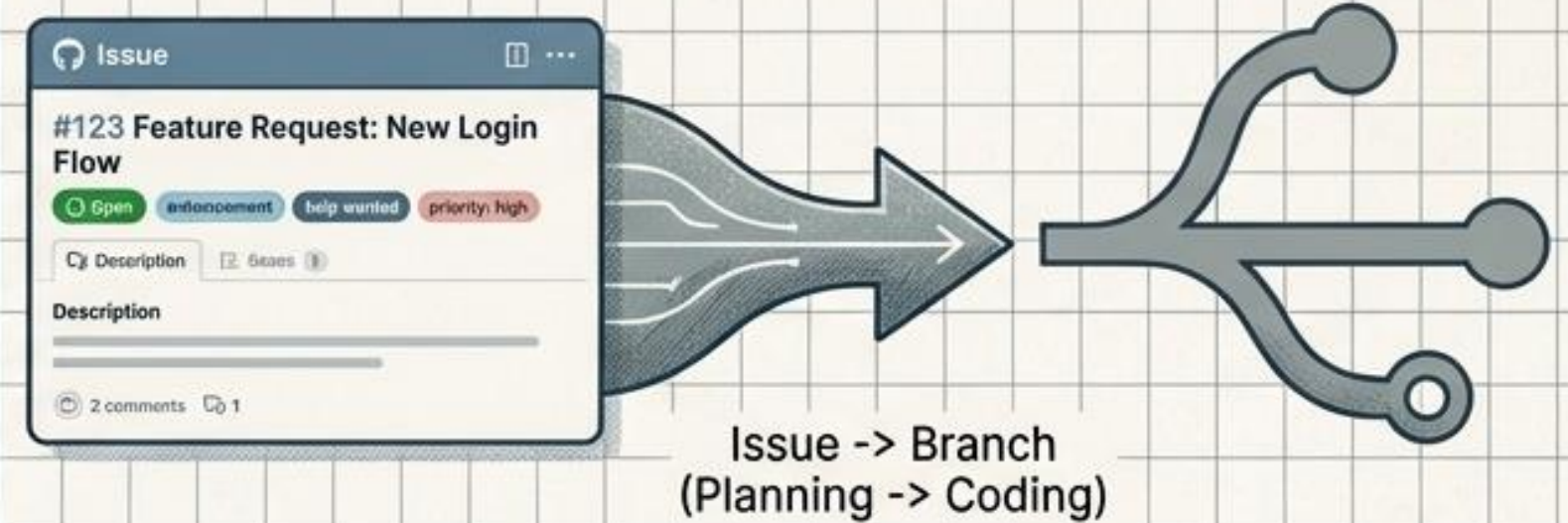
Orchestrating the Work: Issues and Forking

GitHub Issues (Project Management)

Built-in project management.

Used to track bugs, plan features, and assign tasks.

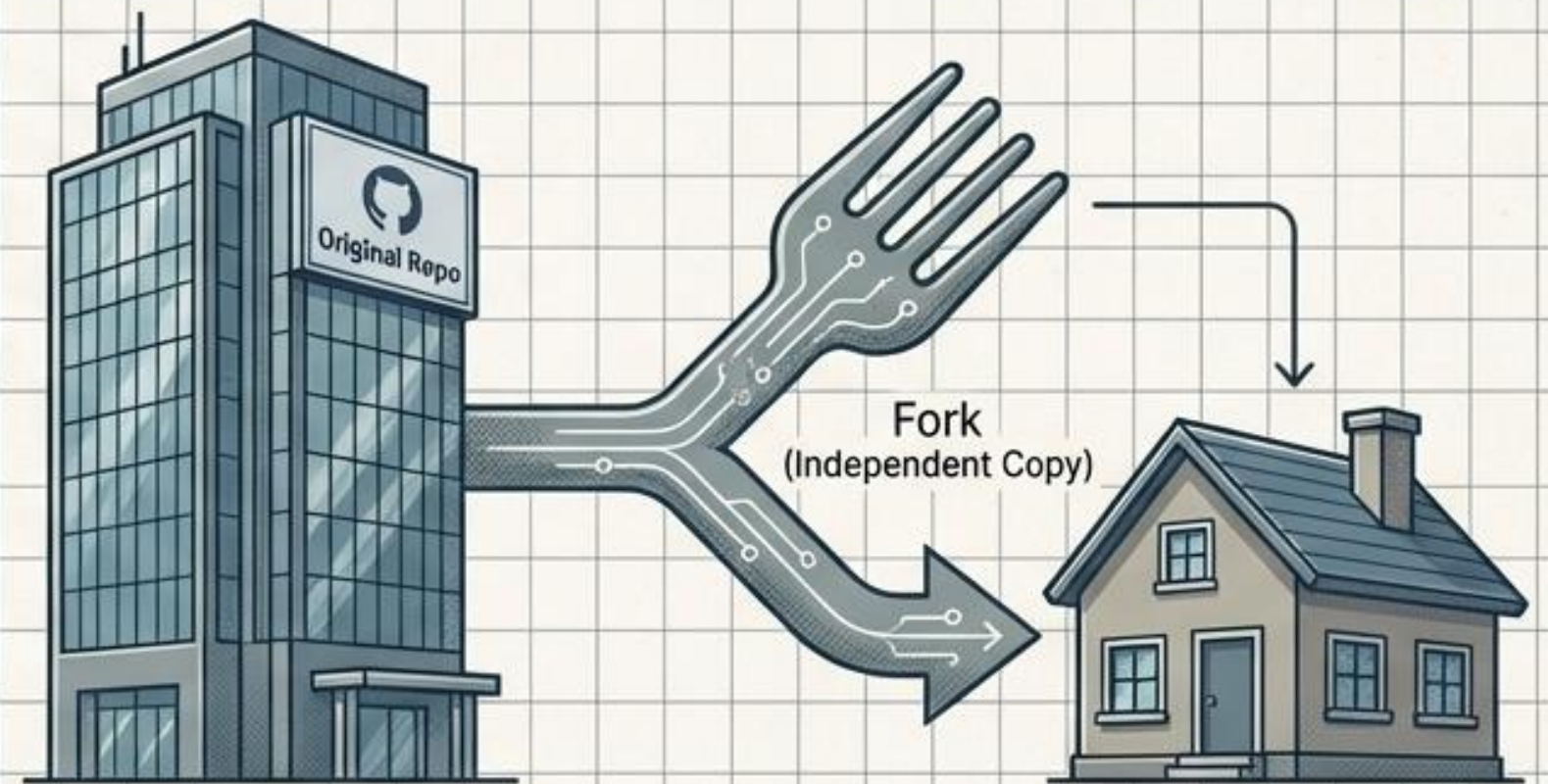
Every Pull Request should reference a specific Issue.



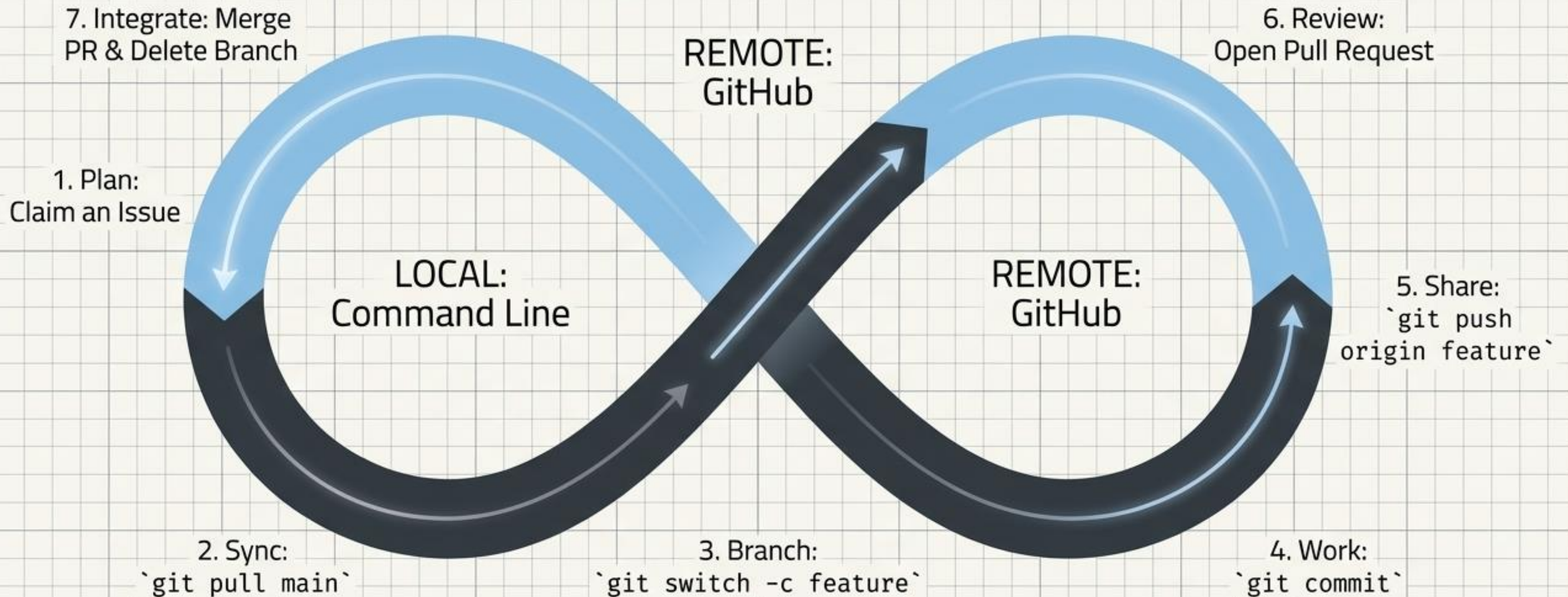
Forking (Open Source)

Creating a personal, independent copy of an external repository you do not own.

Allows contribution to public projects without needing direct write access.



End-to-End: The Seamless Development Loop



Applied Scenario: The Squad Fix

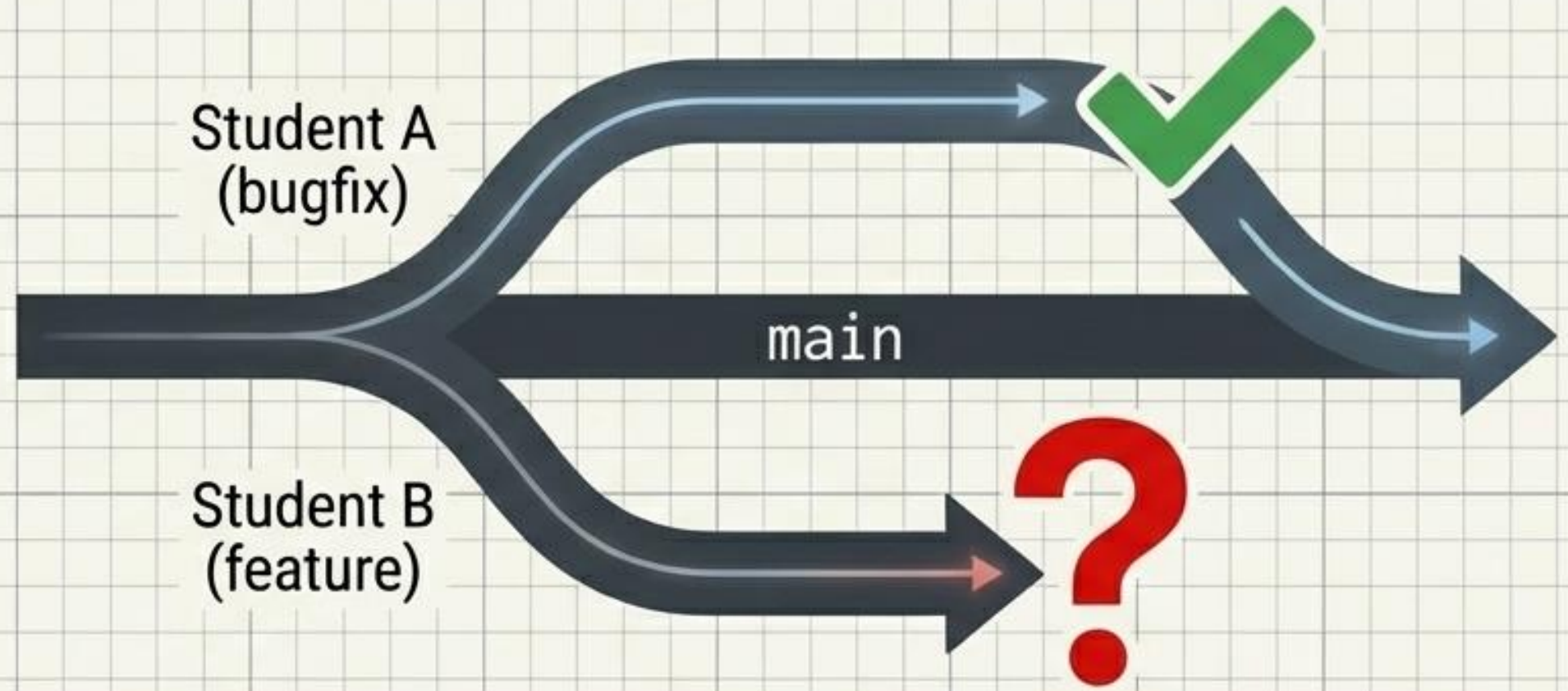
The Setup:

- Student A pulls `main` & creates branch `bugfix`.
- Student B pulls `main` & creates branch `feature`.

The Action:

Student A pushes `bugfix` and merges their PR.

Student B pushes `feature` and attempts to merge their PR. Both edited `index.html` on the exact same line.



Interactive Prompts:

What happens to Student B's Pull Request?

What exact steps must Student B take locally to fix the situation?
